Universidad Tecnológica de Bolívar

Escuela de Transformación Digital · Ingeniería de Sistemas y Computación

Diseño ajustado e implementación de un agente inteligente

Tarea 3 — Segundo Parcial [Agentes Inteligentes]

STREAMRETAIN-AGENT — AGENTE DE PREDICCIÓN DE CANCELACIÓN EN STREAMING

Presentado por:

Christian David Menco Galván — T00058749

Levis Adrian Ortiz Cano — T00083674

Curso: Inteligencia Artificial (ISCO-A06A-I-1233-202620)

Docente: Edwin Alexander Puertas Del Castillo

Cartagena de Indias, 18 de octubre de 2026

Documento base: «Diseño inicial de un agente inteligente — StreamRetain-Agent» (Tarea 2, L. Ortiz, 25 de agosto de 2026), corregido y ampliado.

Tabla de cambios

Retroalimentación del docente: no recibimos retroalimentación del docente para el documento de StreamRetain-Agent de la Tarea 2. Por eso esta tabla recoge **solo los cambios propios** de la pareja: la matriz de decisión, las medidas reformuladas, las precisiones de diseño que surgieron al implementar y la sección 9 nueva. No se incluyen ni se simulan correcciones del docente.

Tabla 1. Cambios respecto al documento de la Tarea 2.

Sección	Qué decía	Qué dice ahora	Por qué cambió
Portada	Presentado por: Levis Adrian Ortiz Cano	Presentado por ambos integrantes de la pareja	La Tarea 3 es en pareja y tiene una sola entrega.
Selección justificada del agente (nueva)	—	Matriz de decisión ponderada, sensibilidad y justificación (Fase 1)	La guía la exige como sección nueva antes del diseño.
4. Descripción del entorno	«El conjunto de datos definitivo deberá quedar formalmente definido antes de la fase de evaluación»	El entorno de evaluación es un simulador estocástico con datos sintéticos, 13 escenarios definidos y congelados antes de probar	No hay acceso a una plataforma real; la guía permite un simulador mínimo que respete las secciones 3 y 5.
5. Entradas y sensores	Estado posterior de la suscripción como dato categórico	Se precisa que el agente lo recibe 4 semanas después de cada predicción y solo lo usa para aprender	Evitar fuga de información: el agente no puede ver el futuro al decidir.
7. Medidas de desempeño	«Precisión general» ≥ 75 % = predicciones correctas / total	Se nombra «exactitud (accuracy)» y se conserva como M1. La medida principal pasa a ser la precisión balanceada ≥ 75 % (M1R). Se agregan M2–M6 con fórmula y umbral	«Precisión» se confunde con «precision». Con tasas de cancelación ≤ 25 %, predecir «nadie cancela» ya da accuracy ≥ 75 %: la medida original no discrimina. La guía pide fórmula, umbral previo y prueba por medida.
8. Razonamiento y toma de decisiones	«Se podrá utilizar SHAP»	Explicación local por oclusión (aporte de cada variable al reemplazarla por su mediana)	Funciona igual con el modelo calibrado, es rápida y evita una dependencia pesada en el despliegue.
8. Aprendizaje y adaptación	Reentrenar si la nueva versión mantiene o mejora la medida	Compuerta concreta: precisión balanceada en un holdout de resultados recientes; se excluyen los usuarios intervenidos	La oferta cambia el resultado de quien la recibe y sesgaría la etiqueta.
9. Desarrollo del agente (nueva)	—	Arquitectura, tecnologías, entradas, salidas, diagramas de secuencia, diferencias, pruebas, limitaciones y enlaces	Exigida por la guía de la Tarea 3.
10. Conclusiones	Sección 9	Pasa a ser la sección 10 y se actualiza con los resultados	Orden exigido por la guía.
Referencias	Formato libre; Russell y Norvig sin año ni edición	Formato APA 7, con citas en el texto que corresponden una a una; se agregan las fuentes usadas en la implementación	Normas del curso.

Selección justificada del agente (Fase 1)

Comparamos los dos agentes que diseñamos en la Tarea 2 con una matriz de decisión ponderada. Agente A: PickFlow-Agent, optimización de picking en bodega (C. Menco). Agente B: StreamRetain-Agent, predicción de cancelación en streaming (L. Ortiz).

Primero fijamos los pesos (los seis criterios mínimos de la guía, con su peso sugerido) y después puntuamos de 1 a 5, con una frase de evidencia por puntaje. El puntaje es $S = \Sigma (w_i \cdot p_i)$, con $\Sigma w_i = 1$ y máximo 5. Todo se calcula con `scripts/matriz_decision.py` a partir de `config/matriz.yaml`, así que cualquiera puede reconstruir la decisión.

Tabla 2. Matriz de decisión ponderada (A = PickFlow-Agent, B = StreamRetain-Agent).

Criterio	Peso	A	Evidencia A	B	Evidencia B
Viabilidad técnica	25 %	3	Exige simular bodega con operarios y robots y aprender una política de asignación y ruta (utilidad + aprendizaje); factible, pero con más componentes nuevos que construir.	5	Pipeline supervisado conocido (scikit-learn: preprocesamiento, clasificador, calibración) más un simulador semanal sencillo; todos los componentes ya los sabemos implementar.
Disponibilidad de datos o entorno	20 %	3	No hay WMS real; el simulador debe modelar rutas, tiempos estocásticos, picos de demanda y fallas de robots, sin datos de referencia para calibrarlo.	4	Las percepciones de la sección 5 (horas vistas, días sin login, plan, cambio de pago) se generan con un simulador semanal estocástico; no existe historial semanal real, por eso no es 5.
Medibilidad del desempeño	20 %	4	El tiempo promedio vs. línea base en simulador es objetivo, pero la medida SMART exige 2 semanas de operación real, que no es replicable y debe reformularse.	5	La medida SMART (precisión general ≥ 75 % = aciertos / predicciones) ya tiene fórmula y umbral, y se calcula sobre datos de prueba no usados en entrenamiento.
Complejidad de IA	15 %	5	Agente de aprendizaje con elemento de desempeño basado en utilidad, ambiente multiagente y secuencial (Russell y Norvig).	3	Agente de aprendizaje con estado interno por usuario, pero la decisión es clasificación supervisada + regla de umbral; menos razonamiento secuencial.
Costo y recursos	10 %	5	Todo corre local en Python con librerías libres; costo estimado COP 0 y sin llamadas a APIs pagas.	5	Todo corre local en Python con librerías libres; la API de marketing es un simulador; costo estimado COP 0.
Valor del problema	10 %	4	Usuario: supervisor de bodega; beneficio: menos tiempo de preparación y desplazamiento (meta de −20 %).	4	Usuario: analista de retención; beneficio: anticipar cancelaciones a 4 semanas y proteger ingreso recurrente.
Puntaje ponderado S	100 %	**3,80**		**4,40**	

Como muestra la Tabla 2, StreamRetain-Agent obtiene **S = 4,40** frente a 3,80 de PickFlow-Agent. Gana en los tres criterios de mayor peso: viabilidad técnica, disponibilidad del entorno y medibilidad. PickFlow solo gana en complejidad de IA, y en costo y valor del problema empatan.

Tabla 3. Análisis de sensibilidad: cada peso ± 10 puntos y el resto reajustado en proporción.

Criterio	Peso original → nuevo	S(A)	S(B)	Gana
Viabilidad técnica (más influyente)	25 % → 15 %	3,91	4,32	B
Viabilidad técnica (más influyente)	25 % → 35 %	3,69	4,48	B
Disponibilidad de datos o entorno	20 % → 10 %	3,90	4,45	B
Disponibilidad de datos o entorno	20 % → 30 %	3,70	4,35	B
Medibilidad del desempeño	20 % → 10 %	3,77	4,33	B
Medibilidad del desempeño	20 % → 30 %	3,83	4,47	B
Complejidad de IA	15 % → 5 %	3,66	4,56	B
Complejidad de IA	15 % → 25 %	3,94	4,24	B
Costo y recursos	10 % → 0 %	3,67	4,33	B
Costo y recursos	10 % → 20 %	3,93	4,47	B

Criterio	Peso original → nuevo	S(A)	S(B)	Gana
Valor del problema	10 % → 0 %	3,78	4,44	B
Valor del problema	10 % → 20 %	3,82	4,36	B

Sensibilidad. El criterio más influyente es la viabilidad técnica, con el mayor $|w \cdot (pA - pB)| = 0{,}50$. Si su peso sube o baja 10 puntos, la decisión **se mantiene** (Tabla 3). Repetimos la prueba con los otros cinco criterios y en ninguna de las 12 variaciones cambia el resultado. El único criterio que podría invertirlo es la complejidad de IA, y para llegar al empate tendría que pesar 35 %, más del doble del peso sugerido.

Qué perdemos al descartar PickFlow-Agent:

- Un ambiente multiagente y secuencial, con decisiones de asignación y ruta que se encadenan.
- Un elemento de desempeño basado en utilidad, más rico en razonamiento que una clasificación con umbral.
- Una medida de impacto operativo directo (−20 % del tiempo de preparación).

Lo compensamos así:

- StreamRetain conserva estado interno por suscriptor.
- Aprende de los resultados con una compuerta de reentrenamiento.
- Prioriza las ofertas con un presupuesto.
- Opera en un entorno estocástico con fallas inyectadas.

1. Introducción y planteamiento del problema

Las plataformas de streaming basadas en suscripción dependen de la permanencia de sus usuarios, por lo que la cancelación del servicio, que se puede denominar como churn, representa un problema relevante para la continuidad de los ingresos y para la planificación de estrategias de retención. En este contexto, el caso de uso StreamRetain-Agent plantea identificar semanalmente a los suscriptores con alta probabilidad de cancelar durante las siguientes cuatro semanas, utilizando señales de comportamiento como las horas vistas, los días transcurridos desde el último inicio de sesión, el tipo de plan y los cambios recientes en el método de pago (Maan & Maan, 2023).

Un agente inteligente resulta pertinente porque no se limita a producir una predicción aislada, sino que percibe periódicamente el estado de cada suscriptor, compara su comportamiento actual con el historial disponible, estima el riesgo de cancelación, prioriza los casos de mayor riesgo y ejecuta una acción sobre el entorno mediante la conexión con el sistema de marketing para activar una oferta de retención. El resultado de cada intervención se registra con el propósito de mejorar decisiones posteriores y conformar un ciclo de percepción, decisión, acción y aprendizaje.

2. Tipo de agente inteligente

El agente se clasifica principalmente como un agente de aprendizaje (Russell & Norvig, 2004). Esta elección se justifica porque su comportamiento no dependerá únicamente de reglas fijas definidas por el diseñador, sino de un modelo entrenado con datos históricos de suscriptores que cancelaron y permanecieron. Además, el agente utilizará un estado interno basado en el historial de cada usuario para interpretar cambios de comportamiento, por ejemplo una disminución sostenida de las horas vistas. Los resultados de las intervenciones de retención se almacenarán para incorporarlos a futuras evaluaciones y reentrenamientos del modelo. Por ello, aunque utiliza características de un agente basado en modelo, su clasificación principal dentro de las opciones de la plantilla es de aprendizaje.

3. Tipos de ambiente

La clasificación sigue las seis dimensiones de Russell y Norvig (2004) y del material del curso (Puertas, 2026).

Tabla 4. Propiedades del ambiente (sin cambios respecto a la Tarea 2).

Dimensión	Clasificación	Justificación
Observabilidad	Parcialmente observable	El agente observa registros de uso, plan, accesos y pagos, pero no puede conocer de forma directa la intención futura del suscriptor ni todos los factores externos que pueden llevarlo a cancelar; debe inferir el riesgo a partir de señales incompletas.
Determinismo	Estocástico	Usuarios con características similares pueden tomar decisiones diferentes y una misma oferta de retención puede producir resultados distintos.
Episodicidad	Secuencial	La evaluación de una semana depende del historial previo del usuario y las acciones ejecutadas pueden influir en su comportamiento posterior.
Dinamismo	Dinámico	El comportamiento cambia con el tiempo (sesiones, consumo, plan, pagos, cancelaciones); el agente actualiza su percepción en cada ciclo semanal.
Discretización	Continuo, con variables discretas	Horas vistas, días desde el último acceso y probabilidad son numéricas; tipo de plan y estado de la suscripción son categóricos.

Dimensión	Clasificación	Justificación
Número de agentes	Agente único	Un solo agente StreamRetain; usuarios y sistema de marketing forman parte del entorno.

4. Descripción del entorno

El entorno operativo corresponde a una plataforma piloto de streaming por suscripción. Los actores principales son los suscriptores activos, el agente StreamRetain y el sistema de marketing o retención. El agente recibe semanalmente la información de comportamiento disponible para cada suscriptor y genera una estimación de riesgo de cancelación a cuatro semanas. Cuando identifica usuarios de mayor riesgo, los prioriza y envía la decisión al sistema de marketing para activar una oferta de retención personalizada. Posteriormente, se registra si el usuario permaneció o canceló.

Como restricciones del dominio, el agente solo puede decidir con base en los datos que la plataforma haya capturado y no debe asumir información histórica inexistente. Entre las condiciones excepcionales que deben manejarse se encuentran los registros faltantes o inválidos, los usuarios sin suficiente historial y los fallos temporales de integración con el sistema de marketing.

Entorno de implementación (corregido). Como no tenemos acceso a una plataforma real, el entorno se implementó como un **simulador estocástico con datos sintéticos** (sección 9.1). El simulador genera las percepciones de la sección 5 y respeta las propiedades de la sección 3. Los 13 escenarios de evaluación (6 normales, 4 límite y 3 de falla esperada) y sus parámetros quedaron definidos y congelados en `config/escenarios.yaml` antes de probar.

5. Entradas y sensores

- **Horas vistas:** dato numérico del nivel de consumo semanal. Captura: registros de reproducción.
- **Días desde el último inicio de sesión:** dato numérico de recencia. Captura: registros de autenticación.
- **Tipo de plan:** dato categórico (básico, estándar, premium). Captura: base de datos de suscripciones.
- **Cambio reciente en el método de pago:** dato binario. Captura: sistema de facturación.
- **Historial semanal de comportamiento:** serie temporal construida con las capturas periódicas para calcular variaciones, tendencias y caídas sostenidas de consumo.
- **Estado posterior de la suscripción e intervención:** dato categórico (permaneció o canceló) que el agente recibe **4 semanas después** de cada predicción, junto con el registro de las ofertas activadas. Solo se usa para aprender, nunca para decidir en la semana en curso (precisión agregada).

6. Salidas y actuadores

- **Probabilidad de cancelación:** riesgo estimado de cancelar en las siguientes cuatro semanas; modifica el estado interno de decisión y permite ordenar prioridades.
- **Clasificación o priorización de riesgo:** organiza a los suscriptores según su riesgo para determinar cuáles requieren intervención.
- **Activación de oferta de retención:** acción ejecutada mediante la llamada al servicio de marketing, que registra al usuario y activa la oferta.
- **Registro de la intervención y su resultado:** escritura en la base histórica de la intervención, la fecha y el resultado observado, que actualiza la memoria del agente.

7. Medidas de desempeño

Medida SMART original (se conserva como M1). Alcanzar una exactitud (accuracy) general mínima del 75 % al identificar, en cada ciclo semanal, a los suscriptores que cancelarán dentro de las siguientes cuatro semanas. Se calcula como predicciones correctas / total de predicciones, sobre datos no usados en el entrenamiento. El 75 % es el umbral definido en el caso de uso.

Reformulación (corregido, antes de probar). Con tasas de cancelación de 25 % o menos, la estrategia trivial de predecir que nadie cancela obtiene una accuracy de 1 − tasa base, es decir, 75 % o más, sin detectar a nadie. Por eso la medida original no permite distinguir un buen agente de uno inútil. La medida principal pasa a ser la **precisión balanceada ≥ 75 %** (M1R), que conserva el umbral del diseño y en la que la estrategia trivial obtiene 50 %. Se agregan cinco medidas complementarias. Todas tienen fórmula y umbral, y se congelaron con el tag git `umbrales-congelados` antes de ejecutar la evaluación.

Tabla 5. Medidas de desempeño y umbrales congelados antes de probar.

ID	Medida	Fórmula	Umbral
M1	Exactitud (accuracy), SMART original	(TP + TN) / N	≥ 75 %
M1R	Precisión balanceada (principal)	(recall + especificidad) / 2	≥ 75 %
M2	Recall de la clase «cancela»	TP / (TP + FN)	≥ 60 %
M3	Precisión en el top-K (K = 10 % de la base)	cancelaciones reales en el top-K / K	≥ 50 %
M4	AUC-ROC	área bajo la curva ROC de la probabilidad	≥ 0,80
M5	Robustez ante entradas inválidas y fallas	fallas manejadas / fallas inyectadas	100 %
M6	Latencia del ciclo semanal (2.000 suscriptores)	segundos por ciclo	≤ 10 s

8. Componentes de la arquitectura de IA agéntica (marco IBM)

Los agentes de inteligencia artificial son sistemas capaces de percibir su entorno, tomar decisiones y ejecutar acciones de manera autónoma. Según la arquitectura de la IA agéntica que describe IBM (s. f.), sus componentes se organizan en siete áreas funcionales.

Percepción y manejo de entradas. El agente recibe y valida los datos semanales de cada suscriptor desde los registros de reproducción, acceso, suscripción y pagos, y los transforma en variables utilizables por el modelo, comparándolos con el historial disponible del propio usuario. Cuando no existe suficiente historial, el sistema reconoce esa limitación en lugar de generar artificialmente observaciones anteriores.

Planificación y descomposición de tareas. En cada ciclo semanal el agente divide su tarea en etapas: recopilar y validar datos, actualizar el historial, calcular cambios de comportamiento, estimar la probabilidad de cancelación, ordenar a los usuarios por riesgo, seleccionar los casos que cumplen el criterio de intervención, ejecutar la acción mediante marketing y registrar el resultado.

Memoria. Historial de observaciones semanales de cada suscriptor, predicciones realizadas, intervenciones aplicadas y sus resultados. Permite construir variación semanal, promedio reciente, tendencia de consumo y semanas consecutivas de descenso, solo con datos realmente capturados.

Razonamiento y toma de decisiones. Un modelo de aprendizaje automático entrenado con ejemplos etiquetados produce una probabilidad de cancelación, que el agente combina con reglas de priorización

(umbral τ y presupuesto de intervenciones). Para la interpretabilidad se usa una **explicación local por oclusión** (corregido; antes: SHAP), que indica qué factores aumentaron o redujeron el riesgo estimado.

Llamada de herramientas y acción (tool calling). Acceso a las fuentes de datos y a la API de marketing: el agente envía los usuarios seleccionados para una acción de retención y almacena el resultado de la intervención.

Comunicación. Presenta al analista la identificación del usuario, su probabilidad, prioridad, confianza y factores principales, y distingue entre una estimación probabilística y una cancelación confirmada.

Aprendizaje y adaptación. Cada predicción se vincula con el estado de la suscripción en las cuatro semanas siguientes. Periódicamente se reentrena el modelo y la nueva versión **solo se acepta si mantiene o mejora la precisión balanceada** en un holdout reciente (corregido: compuerta concreta; se excluyen los usuarios intervenidos).

9. Desarrollo del agente

Esta sección describe la implementación. El código, las instrucciones para ejecutarlo y los logs están en el repositorio (https://github.com/criss132/streamretain-agent). Todos los datos que usa el agente son **sintéticos**: los genera el simulador.

9.1 Arquitectura

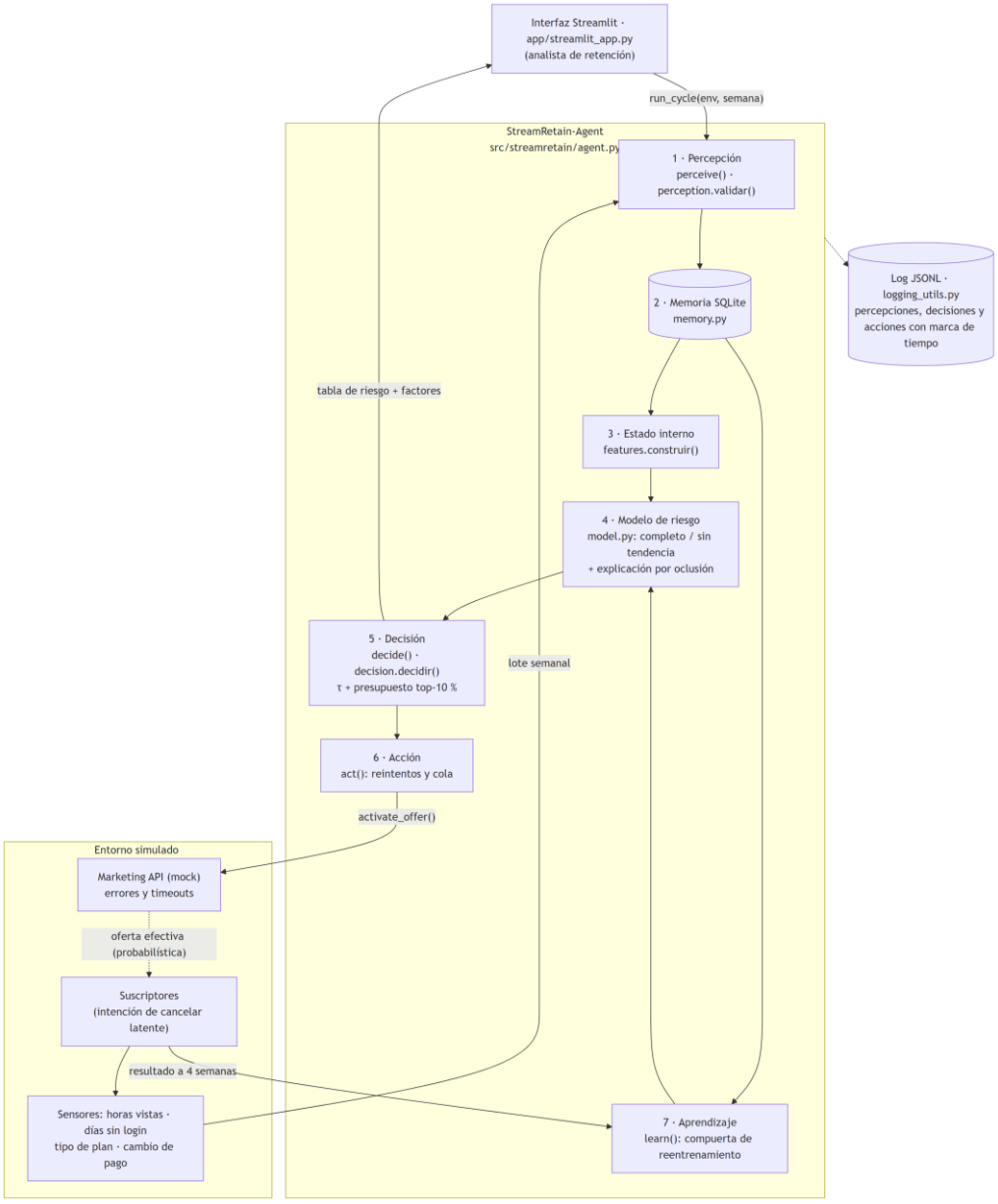


Figura 1. Diagrama de componentes de StreamRetain-Agent y su entorno simulado.

La Figura 1 muestra los componentes y cómo se conectan:

- **Sensores:** el entorno (`StreamingEnv`) entrega cada semana un lote crudo de registros.
- **Percepción:** valida el lote, lo guarda en la memoria SQLite y construye el estado interno de cada usuario (variables estáticas y de tendencia).
- **Modelo de riesgo:** estima la probabilidad de cancelación.

- **Decisión:** aplica el umbral τ y el presupuesto de intervenciones.
- **Acción:** llama a la API de marketing (servicio externo).
- **Aprendizaje:** recibe el resultado a 4 semanas y reentrena el modelo con una compuerta.
- **Interfaz y log:** la interfaz Streamlit orquesta los ciclos y el log JSONL registra cada paso con marca de tiempo.

Tabla 6. Componentes IBM y módulo del código que los implementa.

Componente IBM	Módulo	Estado
Percepción y manejo de entradas	`perception.py` (`validar`), `agent.py` (`perceive`), `features.py` (`construir`)	Implementado
Planificación y descomposición	`agent.py` (`run_cycle`: validar → memoria → variables → predecir → priorizar → actuar → registrar → aprender)	Implementado como descomposición fija. No hay un planificador dinámico porque la tarea semanal es siempre la misma.
Memoria	`memory.py` (SQLite: observaciones, rechazos, predicciones, intervenciones, cola, resultados)	Implementado
Razonamiento y toma de decisiones	`model.py` (`ModeloRiesgo.predecir`, `explicar`), `decision.py` (`decidir`)	Implementado (oclusión en lugar de SHAP)
Llamada de herramientas	`env/simulator.py` (`MarketingAPI.activate_offer`), `agent.py` (`act`)	Implementado contra una API simulada. No hay integración con un sistema real.
Comunicación	`app/streamlit_app.py` (tabla de riesgo, prioridad, confianza y factores), `logging_utils.py`	Implementado. No envía notificaciones a otros sistemas.
Aprendizaje y adaptación	`agent.py` (`learn`, `_reentrenar`), `scripts/train_initial.py`	Implementado

9.2 Tecnologías

Tabla 7. Tecnologías utilizadas.

Tecnología	Versión	Para qué se usó	Por qué frente a una alternativa
Python	3.13.0	Lenguaje de todo el agente	Ecosistema de ML maduro y conocido por la pareja; Java o R exigirían más código propio
NumPy / pandas	2.5.4 / 3.0.6	Simulador vectorizado, variables, tablas de resultados	Estándar para datos tabulares; Polars sería más rápido pero es menos conocido
scikit-learn	1.9.1	Regresión logística, HistGradientBoosting, calibración isotónica, AUC y ROC (Pedregosa et al., 2011)	API uniforme para comparar candidatos; XGBoost no aportaba ventaja a este tamaño de datos
SciPy	1.18.1	Muestreo de los días sin login (Poisson)	Ya viene con scikit-learn
SQLite (sqlite3)	3.45.3	Memoria persistente del agente	Incluido en Python; sin servidor (frente a PostgreSQL)
Streamlit	1.66.0	Interfaz y despliegue (evidencia)	Despliegue gratuito desde GitHub; Flask requiere más código de interfaz
PyYAML	6.0.3	Configuración de escenarios, umbrales y matriz	Legible y versionable (frente a JSON)
matplotlib	3.11.2	Figuras de resultados	Estándar; sin dependencias de navegador
joblib	1.6.0	Guardar modelos y paralelizar la evaluación	Incluido con scikit-learn
pytest	9.1.1	Pruebas unitarias y de integración	Más conciso que unittest
Mermaid	11.4.1	Diagramas de arquitectura y de secuencia	Diagrama como código, versionable (frente a

Tecnología	Versión	Para qué se usó	Por qué frente a una alternativa
			draw.io)
Git / GitHub	—	Repositorio reproducible	Requisito de la guía; integración con Streamlit Cloud

Qué razona el código propio y qué hacen las librerías. El código propio implementa:

- simulador y API simulada;
- validación de percepciones;
- memoria y variables de tendencia;
- regla de decisión (τ, presupuesto, cola y enfriamiento de 4 semanas);
- reintentos;
- vínculo entre predicción y resultado;
- compuerta de reentrenamiento;
- explicación por oclusión;
- evaluación de M1–M6.

scikit-learn solo ajusta los clasificadores y calcula el AUC. **No se usa ningún modelo de lenguaje (LLM).**

9.3 Entradas (inputs)

Tabla 8. Percepciones del agente. Los ejemplos son reales del log `detalle_N2_s1001.jsonl.gz` (N2, semilla 1001, semana 4, 2026-10-18T18:54:25).

Nombre	Tipo	Rango o formato	Origen	Ejemplo del log
user_id	texto	«U» + 5 dígitos	Base de suscripciones (simulador)	U00000
semana	entero	0 – 15	Reloj del ciclo	4
horas_vistas	real	0 – 168 h por semana	Registros de reproducción (sensor)	5,33
dias_desde_ultimo_login	real	0 – 365	Registros de autenticación (sensor)	4
tipo_plan	categórico	básico, estándar, premium	Base de suscripciones	basico
cambio_metodo_pago	binario	0 o 1 (cambio en las últimas 4 semanas)	Sistema de facturación	0
Historial semanal	serie temporal	Ventana de 4 semanas en memoria	Memoria SQLite (propia)	5 semanas con datos válidos (U00006)
Resultado a 4 semanas	binario	0 = permaneció, 1 = canceló	Entorno (estado de la suscripción), 4 semanas después	197 cancelaciones de 1.888 predicciones (semana 7)

Ejemplo real de un registro rechazado en la misma ejecución: `U00048`, con el motivo «dias_desde_ultimo_login faltante o no numérico». No se descarta en silencio: queda en la tabla `rechazos` de la memoria y en el log.

9.4 Salidas y acciones

Tabla 9. Acciones del agente y su efecto.

Acción	Efecto en el entorno o en el agente
Probabilidad de cancelación (`predecir`)	Estado interno: ordena el riesgo. No modifica el entorno.

Acción	Efecto en el entorno o en el agente
Predicción $p \geq \tau$, prioridad (alta, media, baja) y confianza (normal, baja)	Tabla de riesgo para el analista, con 3 factores principales para cada ofertado.
Activar oferta (`MarketingAPI.activate_offer`)	Modifica el entorno: si llega dentro de las 4 semanas previas a una cancelación programada, la evita con probabilidad 30 % (efecto probabilístico).
Reintentar o encolar ante un fallo de la API	Hasta 3 reintentos. Si la falla persiste, el usuario pasa a la cola del ciclo siguiente (la entrada vence a las 4 semanas).
Registrar intervención y resultado	Actualiza la memoria (`intervenciones`, `resultados`), que alimenta el aprendizaje.
Reentrenar con compuerta (`_reentrenar`)	Reemplaza el modelo solo si su precisión balanceada en el holdout es mayor o igual a la del modelo vigente.

9.5 Diagramas de secuencia UML

Los participantes son el analista (actor externo), la interfaz, el agente separado en percepción, decisión/aprendizaje y acción, el entorno, la memoria y la API de marketing (servicio externo). Los mensajes usan los **nombres reales de los métodos** del código. Fuentes en `docs/diagramas/*.mmd`.

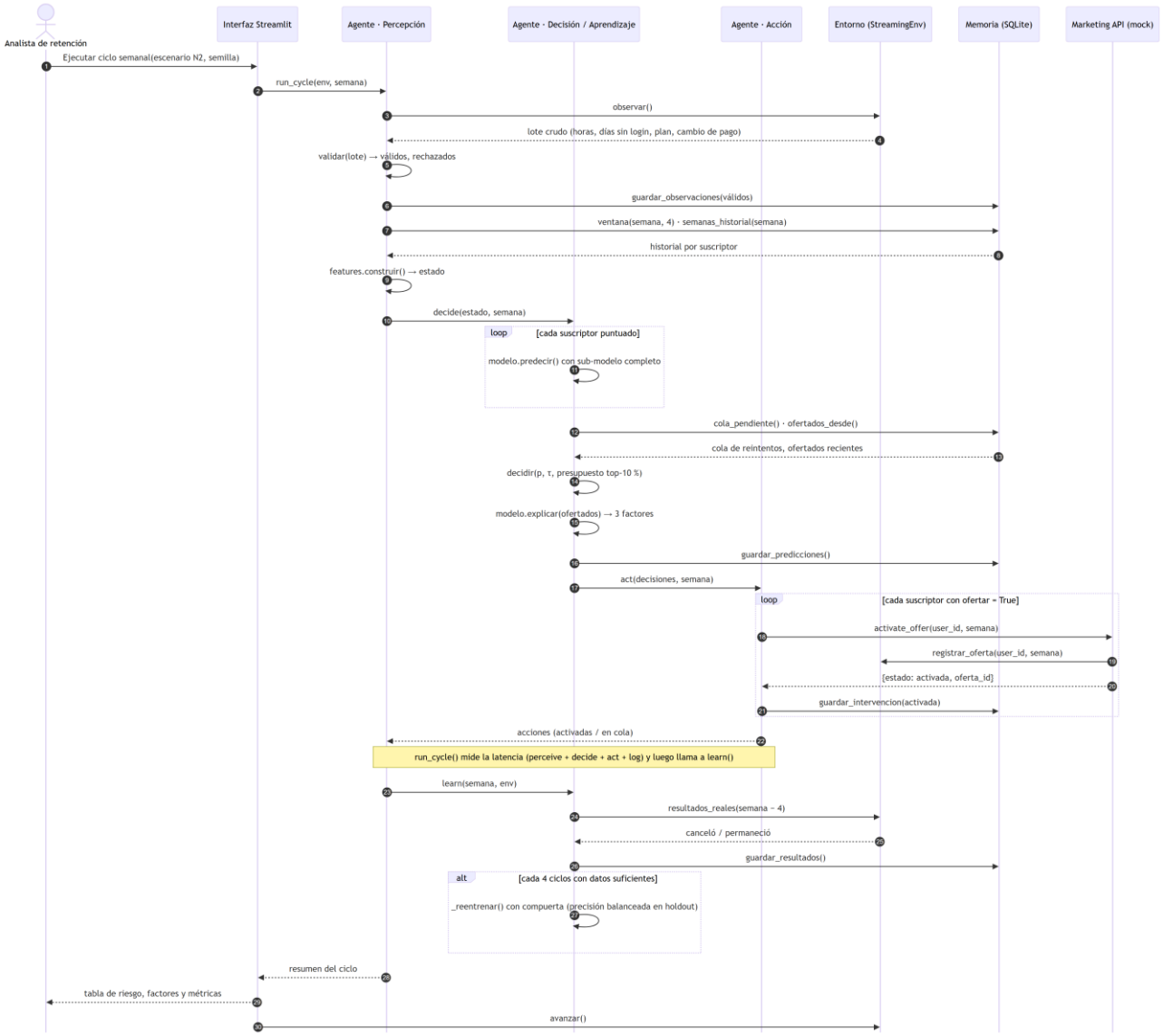


Figura 2. Secuencia del escenario típico (N2): ciclo semanal completo, con fragmentos loop y alt.

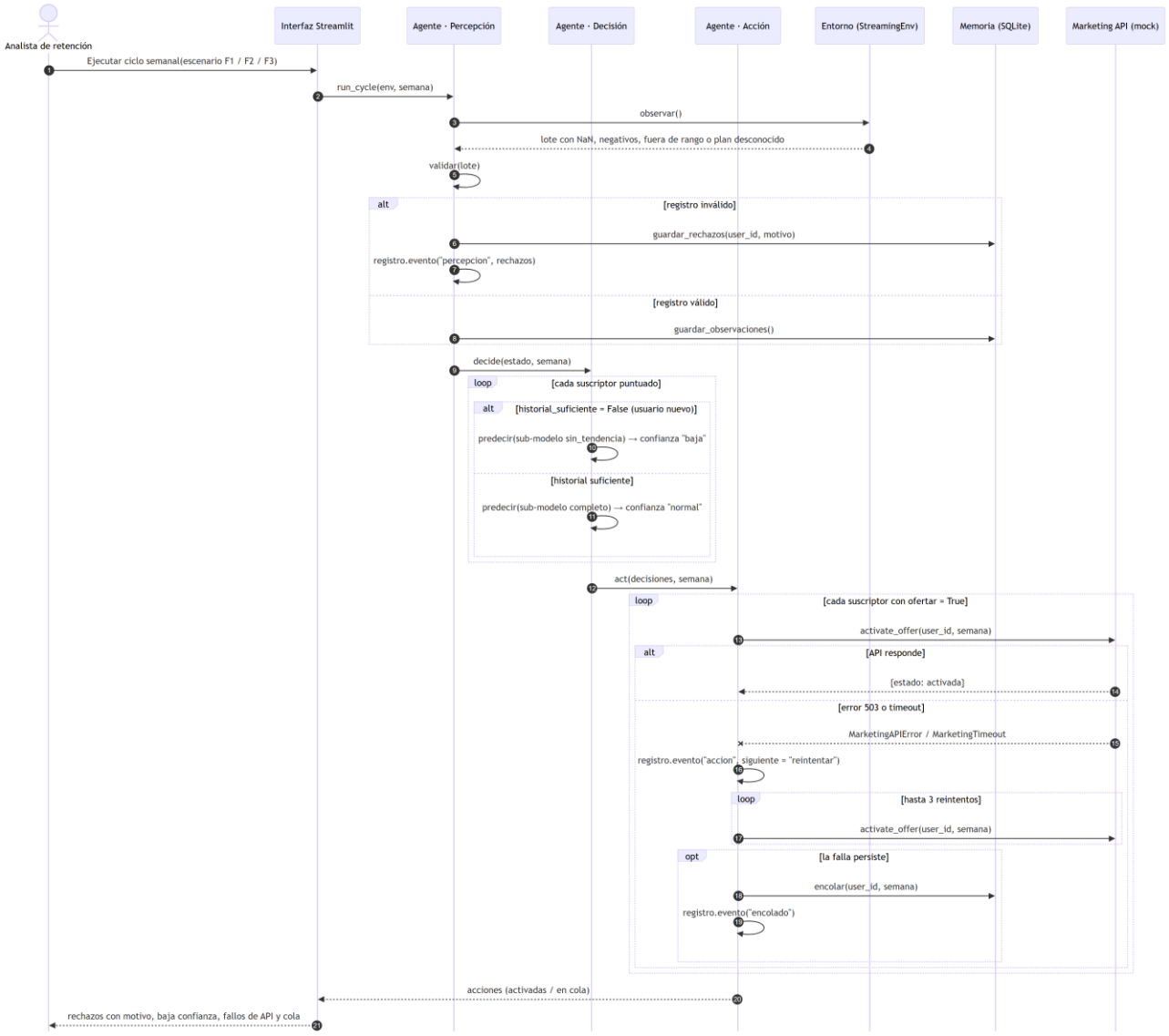


Figura 3. Secuencia del escenario de excepción: registro inválido, usuario sin historial y fallo o timeout de la API, con alt, loop y opt.

La Figura 2 muestra el escenario típico y la Figura 3 los casos de excepción que la guía pide manejar.

9.6 Diferencias entre el diseño y la implementación

Tabla 10. Diferencias entre el diseño y la implementación.

Diseño (Tarea 2)	Implementación	Por qué
Entorno: plataforma piloto real	Simulador estocástico con semilla y datos sintéticos	No hay acceso a datos reales; la guía permite un simulador mínimo.
Entrenamiento con datos históricos etiquetados	«Prehistoria» sintética: 20 poblaciones de entrenamiento (semillas 1–20); modelo y τ elegidos con 5 de validación (501–505)	Sin fuga de datos: las semillas de prueba (1001–1010) se usaron una sola vez.
Explicabilidad con SHAP	Oclusión: aporte = p(x) − p(x con la variable en su mediana)	Funciona con el modelo calibrado, es rápida y aligera el despliegue.
«Reglas de priorización»	τ = umbral de Youden en validación (Youden, 1950); presupuesto del top-10 %; enfriamiento de 4	Hacer verificable y reproducible la regla de intervención.

Diseño (Tarea 2)	Implementación	Por qué
	semanas; cola de reintentos	
Usuarios sin historial: «reconocer la limitación»	Submodelo sin variables de tendencia y confianza «baja»	Seguir prediciendo sin inventar historial.
Aprendizaje con el resultado de la intervención	Se reentrena con los no intervenidos; compuerta con precisión balanceada en un holdout	La oferta altera el resultado de quien la recibe.
Medida: precisión general ≥ 75 %	M1 (se conserva), M1R principal y M2–M6	Ver sección 7.

Transparencia sobre la calibración del simulador. Antes de usar las semillas de prueba encontramos un error: el código sorteaba entre 1 y 3 semanas el inicio de la caída de consumo, cuando el parámetro congelado `semanas_decaimiento_previo` fija 3 semanas. Lo corregimos y revertimos un ajuste de ruido que habíamos hecho con el código erróneo. Con los valores iniciales se cumple el criterio de dificultad congelado: AUC de la regresión logística = 0,7504, dentro de 0,70–0,90, medido solo con semillas de entrenamiento. Todo quedó registrado en el historial de git y en la bitácora de IA.

9.7 Pruebas

Protocolo. Probamos 13 escenarios × 10 semillas de prueba = **130 casos**: 6 normales (tasas base de 10, 15 y 20 %, base de 500 y 2.000 suscriptores, 12 ciclos y otra mezcla de planes), 4 límite (1 semana de historial, tasas de 3 % y 40 %, deriva de comportamiento) y 3 de falla esperada (10 % de registros inválidos, 20 % de usuarios nuevos sin historial, API con 30 % de fallos). En cada caso el agente arranca desde cero y corre 8 ciclos (12 en N5). M1–M4 se miden contra la cancelación «sin intervención» que conoce el simulador; M5 compara las fallas inyectadas con lo que registró el agente. Cada medida se reporta como media ± desviación estándar sobre sus casos.

Tabla 11. Tabla de resultados (obligatoria). Fuente: `results/resultados.csv`; logs en `logs/eval_20261018_1853_2dbc2c8/`.

Medida de desempeño	Fórmula	Umbral	Casos ejecutados	Resultado obtenido	¿Cumple?	Evidencia (CSV y logs)
M1: Precisión general (accuracy), medida SMART original	(TP + TN) / N, en cada ciclo semanal, agrupado por caso	≥ 75,0 %	130	74,5 % ± 1,2 %	**No**	detalle_por_caso.csv (M1_accuracy); logs detalle_*_s1001.jsonl.gz
M1R: Precisión balanceada (balanced accuracy), medida SMART reformulada	(recall + especificidad) / 2 = (TP/(TP+FN) + TN/(TN+FP)) / 2	≥ 75,0 %	130	69,3 % ± 0,9 %	**No**	detalle_por_caso.csv (M1R_balanced_accuracy); logs detalle_*_s1001.jsonl.gz
M2: Recall de la clase 'cancela'	TP / (TP + FN)	≥ 60,0 %	130	61,1 % ± 2,5 %	**Sí**	detalle_por_caso.csv (M2_recall); logs detalle_*_s1001.jsonl.gz
M3: Precisión en el top-K (K = 10 % de la base activa)	cancelaciones reales dentro del top-K / K	≥ 50,0 %	120	57,7 % ± 12,7 %	**Sí**	detalle_por_caso.csv (M3_precision_top_k); logs detalle_*_s1001.jsonl.gz
M4: AUC-ROC	área bajo la curva ROC de la probabilidad predicha (independiente de tau)	≥ 0,800	130	0,761 ± 0,011	**No**	detalle_por_caso.csv (M4_auc); logs detalle_*_s1001.jsonl.gz
M5: Robustez ante entradas inválidas y fallas del entorno	eventos de falla manejados correctamente / eventos de falla inyectados	≥ 100,0 %	130	100,0 % ± 0,0 %	**Sí**	detalle_por_caso.csv (M5_robustez); logs detalle_*_s1001.jsonl.gz
M6: Latencia del ciclo semanal	segundos por ciclo (perceive + decide + act + log), media por caso	≤ 10,00 s	120	1,73 s ± 0,39 s	**Sí**	detalle_por_caso.csv (M6_latencia_media_s); logs detalle_*_s1001.jsonl.gz

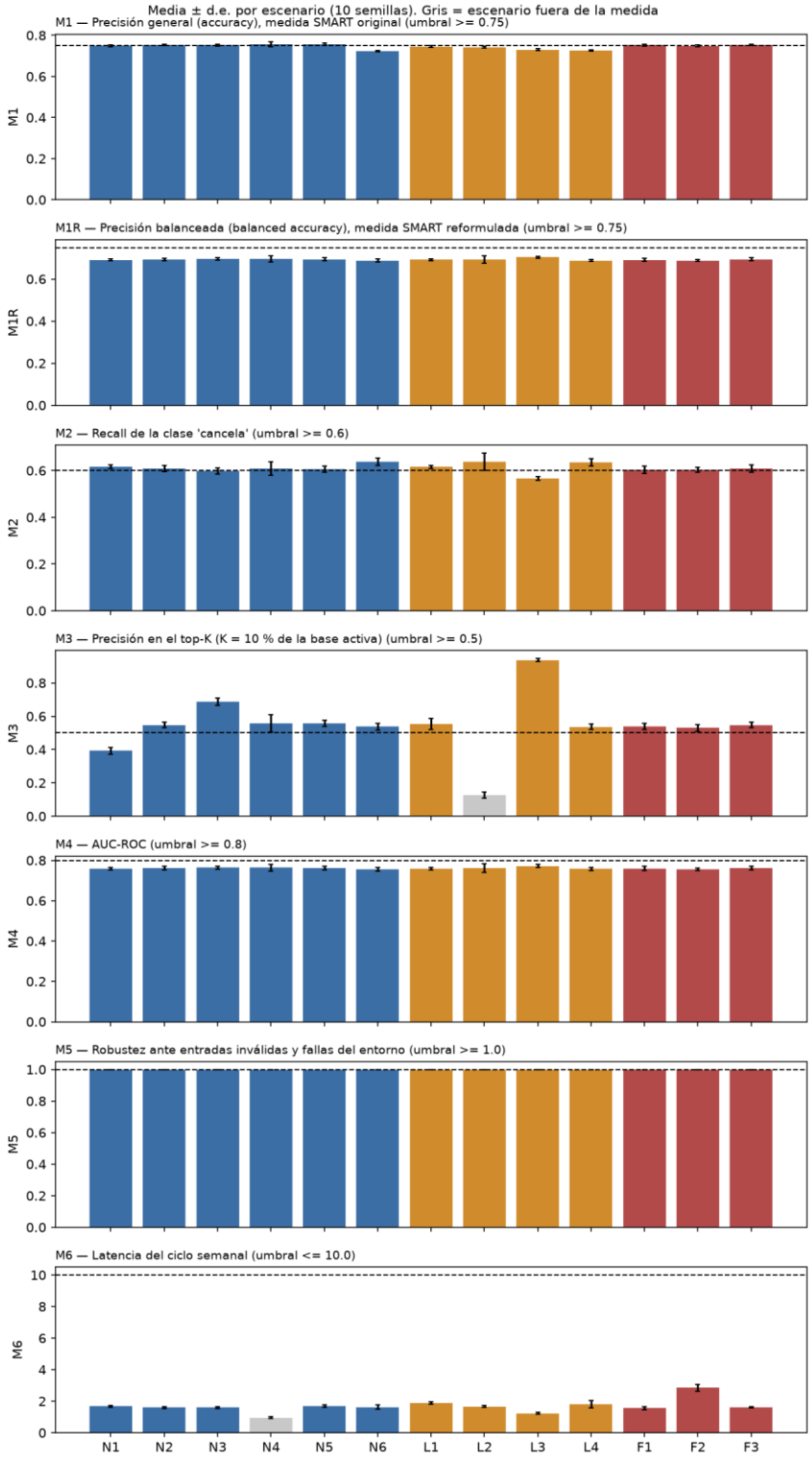


Figura 4. Media ± d.e. de cada medida por escenario (10 semillas). Línea punteada = umbral; gris = escenario fuera de la medida.

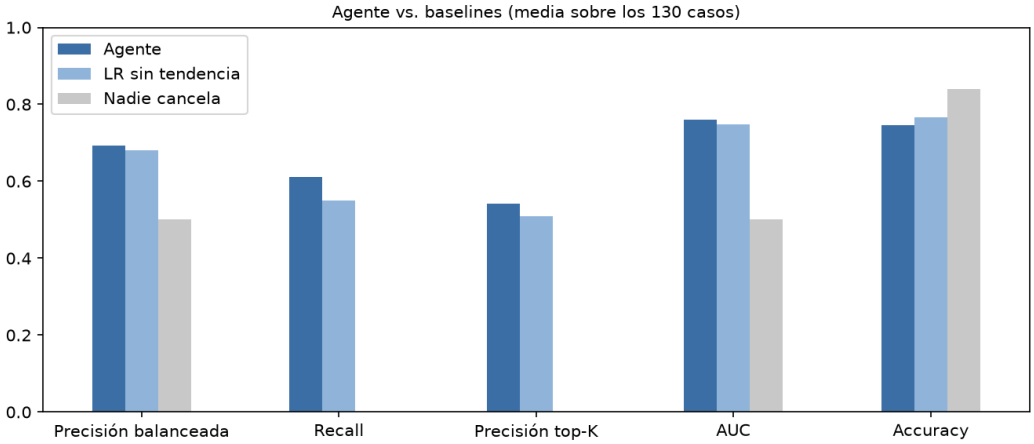


Figura 5. Agente frente a los baselines (media de los 130 casos).

Análisis de resultados. Se cumplen **4 de 7** medidas (Tabla 11, Figuras 4 y 5).

Lo que se cumplió:

- **M5 (robustez):** cero excepciones en 130 ejecuciones. Se manejaron los 51.194 registros inválidos (rechazados con motivo), las 68.640 predicciones sin historial (baja confianza, sin historial inventado) y los 12.441 fallos o timeouts de la API (reintento o cola).
- **M6 (latencia):** 1,73 s de media (escenarios de 2.000 suscriptores) y 3,65 s como máximo.
- **M2 (recall):** se cumple por poco (61,1 %).
- **M3 (precisión en el top-10 %):** se cumple en promedio, pero depende de la tasa base: 39,3 % en N1 (tasa 10 %) frente a 93,7 % en L3 (tasa 40 %).

Lo que no se cumplió y por qué:

- **M1 (74,5 %).** Queda por debajo del baseline «nadie cancela» (83,9 % de media). Es el efecto que anticipamos al reformular: τ acepta falsos positivos para detectar a quien sí cancela.
- **M1R (69,3 %) y M4 (0,761).** La causa principal es la falta de señal. Solo el **57,4 %** de quienes cancelan en las 4 semanas siguientes mostraba alguna señal cuando se hizo la predicción. Los demás todavía no habían decidido cancelar (la caída empieza 3 semanas antes de la baja) o cancelan sin aviso previo.
- El agente detecta al 80,0 % de los que tienen señal y solo al 35,8 % de los que no.
- El AUC es casi idéntico en los 13 escenarios (d.e. 0,011): el techo lo pone la información de los sensores, no un error del modelo.

Frente a los baselines, el agente supera a la regresión logística sin tendencia en precisión balanceada (69,3 % frente a 68,1 %), recall (61,1 % frente a 55,1 %) y AUC (0,761 frente a 0,747). La memoria y las variables de tendencia aportan una mejora real pero modesta. La compuerta de aprendizaje aceptó 58 reentrenamientos y rechazó 82.

Qué cambiaríamos:

- Sumar sensores que anticipen la decisión: visitas a la página de cancelación, tickets de soporte o pagos rechazados.
- Alinear el horizonte de predicción con la señal: predecir a 2 semanas.
- Usar un τ adaptativo a la tasa base para no perder recall cuando sube la cancelación (L3).
- Construir una línea base individual con más de 4 semanas de memoria.

9.8 Limitaciones y trabajo futuro

- Los datos son **sintéticos**: los resultados describen el agente en el simulador, no en una plataforma real. El siguiente paso es validarlo con datos reales anonimizados.
- La evaluación usa la etiqueta «sin intervención», que solo conoce el simulador. En producción haría falta un **grupo de control** sin ofertas.
- τ y el modelo se eligieron con la tasa de referencia del 15 %; con tasas muy distintas cae el recall (L3) o la precisión en el top-K (N1, L2).
- El efecto de la oferta (30 %) es un supuesto del simulador; habría que medirlo con experimentos A/B.
- Aspectos éticos pendientes del diseño original: privacidad, uso proporcional de los datos, sesgos por plan y efectos de ofrecer beneficios distintos a distintos usuarios.
- Trabajo futuro: SHAP como explicación complementaria, horizonte de 2 semanas, nuevos sensores y aprendizaje en línea con detección de deriva (L4).

9.9 Enlaces

- **Repositorio:** https://github.com/criss132/streamretain-agent
- **Plataforma desplegada (Streamlit Community Cloud):** https://streamretain-agent.streamlit.app/
- **Logs de las pruebas:** `logs/eval_20261018_1853_2dbc2c8/` y `results/detalle_por_caso.csv`
- **Bitácora de uso de IA:** `docs/BITACORA_IA.md`

10. Conclusiones

StreamRetain-Agent quedó implementado como un agente de aprendizaje con estado interno, con el ciclo percepción → decisión → acción → aprendizaje explícito en el código. Percibe y valida registros semanales, mantiene memoria por suscriptor, estima el riesgo con un modelo calibrado, prioriza con un umbral y un presupuesto, actúa sobre una API externa con manejo de fallas y aprende de los resultados con una compuerta que impide aceptar modelos peores. Todo funciona sobre un simulador estocástico que respeta el ambiente declarado en la sección 3.

Las pruebas, con umbrales congelados antes de ejecutarlas, muestran un agente robusto y rápido: M5 = 100 % y M6 = 1,73 s. También cumple el recall y la precisión en el top-K. En cambio, no alcanza la meta de discriminación: precisión balanceada de 69,3 % y AUC de 0,761. El análisis apunta a un límite de información y no a un defecto de implementación: solo el 57,4 % de las cancelaciones muestra alguna señal al momento de predecir.

La lección para la siguiente fase es que la meta del 75 % depende más de qué se observa que de qué modelo se usa. Se necesitan sensores que anticipen la decisión o un horizonte de predicción alineado con la señal. Siguen pendientes los aspectos éticos del diseño original: privacidad, proporcionalidad en el uso de datos, sesgos y equidad de las ofertas.

Referencias

IBM. (s. f.). *Components of AI agents*. IBM Think. https://www.ibm.com/mx-es/think/topics/components-of-ai-agents

Maan, J., & Maan, H. (2023). Customer churn prediction model using explainable machine learning. *International Journal of Computer Science Trends and Technology, 11*(1), 33–38.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Puertas, E. (2026). *Inteligencia artificial: Agentes inteligentes* [Material de clase]. Universidad Tecnológica de Bolívar.

Russell, S. J., & Norvig, P. (2004). *Inteligencia artificial: Un enfoque moderno* (2.ª ed.). Pearson Educación.

Youden, W. J. (1950). Index for rating diagnostic tests. *Cancer, 3*(1), 32–35.